第 3 次作业

第 3 章习题

3.1 对一个栈的输入序列 $a_1, a_2, a_3, \ldots, a_n$,，称由此栈依次出栈后所得到的元素序列为栈的合法输出序列。例如，假设栈 S 的一个输入序列为 1,2,3,4,5，则可得到多个输出序列，例如，1,2,3,4,5 就是一个合法的输出序列，同理，5,4,3,2,1 和 3,2,1,4,5 也分别是其合法的输出序列。分别求解下列问题：

（1）判断序列 1,3,4,5,2 是否是合法的输出序列。

（2）对输入序列 1,2,3,4,5，求出其所有的合法的输出序列。

（3*）设计算法以判断对输入序列 1,2,3,…,n，序列 $a_1, a_2, a_3, \ldots, a_n$ 是否是该栈的合法的输出序列（假设输出序列在数组 A 中）。

【解】

（1）手工判断栈的输出序列的合法性是各类考试中经常出现的题型。我们用一个更具典型性的序列来介绍判定方法，假设入栈顺序为 1、2、3、4、5，判定序列 4、3、5、2、1 的合法性。

从左往右依次考察输出序列中的每个数字，序列中 4 为第一个出栈数字，说明比它小的数字 1、2、3 一定在栈中，此后无论怎样操作一定会是 3 出栈、再 2 出栈、再 1 出栈，即比当前数字 4 小的数字，不在其左边出现的话，在其右边一定会按降序出现，即一定先出现 3，后出现 2，再出现 1。次输出序列中 3、2、1 在 4 的右边降序出现合法。再分析输出序列中第二个数字 3，比它小的数字 1、2 没有出现在 3 的左边，在其右边降序出现，也合法。再检查 5，比它小的 1、2、3、4，其中 4、3 出现在其左边，无关升降，1、2 在其右边降序合法。再看 2，只有 1 比它小，在其右边，也合法。所以，此序列是栈的一个合法输出序列。

总结判定方法：对给定的输出序列，从左往右依次检查序列中的每个数字。对当前检查数字，比它小的数字，或者出现在这个数字的左边；如果出现在其右边，这些数字一定按降序出现，为合法。如果输出序列中每个数字都满足这个条件即为合法序列，否则非法。

判定序列 4、3、5、1、2 是否合法序列？对 4，比它小的 1、2、3 均出现在其右边，但 1、2 不是降序，所以此序列非法。

再来分析题中给出的序列 1、3、4、5、2。对 1 没有比它小的数；对 3，比它小的 1 出现在其左边，还有一个 2 在其右边，合法；对 4，1、3 在其左，2 在右，合法；对 5，1、3、4 在左，只有 2 在其右，合法，所以此为栈的一个合法输出序列。其出入栈操作序列为 push(1)pop(1)push(2)push(3)pop(3)push(4)pop(4)push(5)pop(5)pop(2)。

（2）对 n 个元素的入栈序列，合法的出栈序列数为 Catalan 数，等于

$$\frac{C_{2n}^{n}}{n+1}=\frac{(2n)!}{(n+1)\times n!\times n!}=\frac{(2n)!}{(n+1)!\times n!}$$

本题中入栈序列 5 个元素，可以计算出合法的出栈序列数为 42 个。

对于个数较少的入栈序列可用合法的 push 和 pop 操作来枚举出所有的合法出栈序列。如果入栈序列元素较多，要使用栈和递归技术编写程序来输出所有出栈序列。

【一种求解算法描述】

```
//求解所有可能出栈序列
void legalSequence(Stack<int,100> S, elementType In[], elementType Out[], int len, int i, int j)
{
    //In[]入栈序列，也可以用栈来保存，但要注意顺序。还可用 C++的 vector 保存。
    //Out[]出栈序列，也可以用栈来保存，但要注意输出循序。还可用 C++的 vector 保存。
    //len 序列长度
    //i 入栈序列元素指针，指示当前处理的入栈序列元素
    //j 出栈序列元素指针，指示当前获取的出栈序列元素

    //每次操作有可能有 2 中操作，要么出栈，要么入栈，且 2 种操作是或的关系。但，出栈、入栈递归返回后要恢复递归前状态

    elementType x;

    if(S.empty() && j>=len)  //递归出口，获得了一个出栈序列
    {
        seqNum++;            //序列数加 1
        seqPrint(Out, len);  //打印序列
    }
    else if(!S.empty() && i<len)  //栈不空，入栈序列中还有数据
    {
            //选择出栈
        S.getTop(x);
        S.pop();
        Out[j]=x;
        j++;
        legalSequence(S, In, Out, len, i, j);
        j--;  //递归返回，恢复到出栈前的状态
        S.push(x);
            //选择入栈
        S.push(In[i]);
        i++;
        legalSequence(S, In, Out, len, i, j);
        i--;     //恢复到入栈前的状态
        S.pop();
    }
    else if(!S.empty() && i>=len)  //栈不空，入栈序列数据已经处理结束
    {
```

```
            //此时只能选择出栈操作
        S.getTop(x);
        S.pop();
        Out[j]=x;
        j++;
        legalSequence(S, In, Out, len, i, j);
        j--;   //恢复到出栈前状态
        S.push(x);
    }
    else if(S.empty() && i<len)  //栈空，入栈序列未处理结束
    {
            //此时，只能选择入栈操作
        S.push(In[i]);
        i++;
        legalSequence(S, In, Out, len, i, j);
        i--;   //恢复到入栈前的状态
        S.pop();
    }

}
```

（3）用数组 In[]保存入栈序列，Out[]保存输出序列，分别用指针 i，j 指示。用一个栈模拟进出站的操作。对每个输出元素 Out[j]做如下操作：

如果 In[i]<Out[j]，将 In[i]入栈，指针 i 后移，即 i++；

如果 In[i]==Out[j]，2 个指针同时后移，即 i++，j++；

如果栈顶元素==Out[j]，栈顶元素出栈，j 后移，即 j++；

循环结束后，如果栈空，且入栈序列处理完毕，则输出序列是一个合法的出栈序列，否则不是。

【算法描述】

```
bool legalSequence(elementType In[], elementType Out[], int len)
{
    //In[]保存入栈序列
    //Out[]保存输出序列
    int i=0,j=0;  //i 为入栈序列指针；j 为输出序列指针
    elementType x;
    Stack<int,100> S;  //初始化顺序栈
    while(j<len)
    {
        S.getTop(x);
        if(In[i]<Out[j] && i<len)  //入栈序列当前元素小于输出序列当前元素
        {
```

```
            S.push(In[i]);    //输入序列当前元素入栈
            i++;              //入栈序列指针后移
        }
        else if(In[i]==Out[j])    //入栈序列当前元素等于输出序列当前元素
        {
            i++;   //指针同时后移
            j++;
        }
        else if(x==Out[j])   //栈顶元素等于输出序列当前元素
        {
            S.pop();     //出栈
            j++;         //输出序列指针后移
        }
        else
            break;
    }
    if(S.empty() && i>=len)  //栈空，且入栈序列处理完毕，则输出序列为合法出栈序列，返回 true
        return true;
    else
        return false;
}
```

3.5 对表达式 5+3*(12+4)/4-8，依次画出在求解过程中的各步骤中的栈的状态。

【解】

构建 2 个栈，分别用于保存运算符（含定界符）和操作数。以'#'作为整个表达式的定界符。

3.9 对教材中所讨论的循环队列及其约定，给出求解队列中元素个数的表达式。

【解】

设循环队列为 Q，数组最大长度为 MaxLen，头指针和尾指针分别为 front 和 rear

元素个数=(Q.rear-Q.front+MaxLen) % MaxLen

3.11 如果采用带尾指针的单循环链表（带头结点）作为队列的存储结构，设计算法以实现队列的各运算。

【解】

```
//带尾指针单循环链表实现链队列，以下 Q 为单循环链表尾指针
#include "..\scrLinkedList.h"      //带尾指针单循环链表
typedef linkedList linkedQueue;
//--1.链队初始化-----------------------------------------------
void initialQueue(linkedQueue &Q)
```

```
{
    Q=new node;   //产生头结点，指针为 R;
    Q->next=Q;
}
//--2.判断队空-----------------------------------------------
bool queueEmpty(linkedQueue &Q)
{
    return (Q==Q->next);
}
//--3.取队头元素-----------------------------------------------
bool getFront(linkedQueue &Q, elementType &x)
{
    if(queueEmpty(Q))
        return false;   //空队列，无法取队头元素
    else
    {
        x=(Q->next->next)->data;
        return true;
    }
}
//--4.入队-----------------------------------------------------
void enQueue(linkedQueue &Q, elementType x)
{
    node* p=new node;   //申请内存，产生新节点
    p->data=x;
    p->next=Q->next;
    Q->next=p;
    Q=p;
}
//--5.出队-----------------------------------------------------
bool outQueue(linkedQueue &Q, elementType &x)
{
    node* u; //用以指向删除节点
    if(queueEmpty(Q))
        return false;      //队空，无法执行出队操作
    else
    {
        x=Q->next->next->data;      //取出队头元素
        u=Q->next->next;            //u 指向队头
        Q->next->next=u->next;
        if(Q==u)     //删除最后一个节点，成为空队列
```

```
            Q=Q->next;
        delete u;                    //删除原队头，释放内存
        return true;
    }
}
```